

# C++ 프로그래밍

C에서 C++로 넘어가기

# C++는 C의 상위 집합 (대부분)

C++은 C를 기반으로 설계되어, 대부분의 C 코드는 C++ 컴파일러로 그대로 컴파일된다.

```
1  C 언어
2
3  변수, 포인터, 배열, 구조체, 함수, 전처리기 ...
4
5  C++17 추가 기능
6
7  클래스, 참조, 템플릿, 예외, 네임스페이스, 오버로딩,
8  new/delete, STL, 스마트 포인터,
9  string_view, optional, filesystem, byte,
10 구조적 바인딩, if constexpr, inline 변수 ...
11
12
```

"대부분"인 이유: C99/C11의 일부 기능(가변 길이 배열 등)은 C++ 표준에서 지원하지 않는다.

```
1  // 이 C 코드는 C++17 컴파일러에서도 그대로 동작한다
2  #include <stdio.h>
3  #include <stdlib.h>
4
5  int main(void) {
6      int* arr = (int*)malloc(5 * sizeof(int));
7      arr[0] = 42;
8      printf("%d\n", arr[0]);
9      free(arr);
10     return 0;
11 }
```

# C 헤더를 C++에서 포함하기

C 표준 라이브러리 헤더는 C++에서 두 가지 방식으로 포함할 수 있다.

## C 스타일 헤더 (<xxx.h>)

```
// C 원본 헤더 - C++에서도 동작함
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <math.h>

int main() {
    printf("hello\n");           // 전역 네임스페이스
    int* p = (int*)malloc(4);
    free(p);
}
```

- 함수가 **전역 네임스페이스**에 배치됨
- C++에서도 동작하지만 권장하지 않음
- C와의 호환성을 위해 표준에 유지됨

## C++ 래퍼 헤더 (<cxxx>)

```
// C++17 - std 네임스페이스에 배치
#include <cstdio>
#include <cstdlib>
#include <cstring>
#include <cmath>

int main() {
    std::printf("hello\n");      // std:: 네임스페이스
    int* p = (int*)std::malloc(4);
    std::free(p);

    double r = std::sqrt(2.0);
    std::printf("%.4f\n", r);
}
```

- 함수가 **std 네임스페이스**에 배치됨
- C++17 코드에서 **권장하는 방식**
- **using namespace std;** 후엔 접두사 없이 사용 가능

# C 헤더 대응표

| C 헤더                          | C++ 헤더                       | 주요 내용                                                                               |
|-------------------------------|------------------------------|-------------------------------------------------------------------------------------|
| <code>&lt;stdio.h&gt;</code>  | <code>&lt;cstdio&gt;</code>  | <code>printf</code> , <code>scanf</code> , <code>fopen</code> , <code>fclose</code> |
| <code>&lt;stdlib.h&gt;</code> | <code>&lt;cstdlib&gt;</code> | <code>malloc</code> , <code>free</code> , <code>exit</code> , <code>atoi</code>     |
| <code>&lt;string.h&gt;</code> | <code>&lt;cstring&gt;</code> | <code>strcpy</code> , <code>strlen</code> , <code>memcpy</code>                     |
| <code>&lt;math.h&gt;</code>   | <code>&lt;cmath&gt;</code>   | <code>sqrt</code> , <code>pow</code> , <code>sin</code> , <code>cos</code>          |
| <code>&lt;time.h&gt;</code>   | <code>&lt;ctime&gt;</code>   | <code>time</code> , <code>clock</code> , <code>difftime</code>                      |
| <code>&lt;assert.h&gt;</code> | <code>&lt;cassert&gt;</code> | <code>assert</code>                                                                 |
| <code>&lt;errno.h&gt;</code>  | <code>&lt;cerrno&gt;</code>  | <code>errno</code> , <code>strerror</code>                                          |
| <code>&lt;limits.h&gt;</code> | <code>&lt;climits&gt;</code> | <code>INT_MAX</code> , <code>CHAR_MIN</code> 등                                      |

## 실제 사용 예

```
#include <cstdio>
#include <cstring>
#include <cmath>

char buf[64];
std::strcpy(buf, "hello");
```

## using 선언으로 편의성 향상

```
#include <cstdio>
#include <cmath>

// 자주 쓰는 함수만 선택적으로 가져오기
using std::printf;
using std::sqrt;
```

# std::string — C++ 문자열

C의 `char` 배열 대신 `std::string` 을 사용하면 메모리를 자동으로 관리하고 다양한 연산을 편리하게 쓸 수 있다.

## 선언과 초기화

```
#include <string>

std::string s1;                // 빈 문자열
std::string s2 = "hello";      // 문자열 리터럴로 초기화
std::string s3("world");       // 생성자 호출
std::string s4(5, 'A');        // "AAAAA"
std::string s5 = s2;           // 복사
```

## 기본 정보 조회

```
std::string s = "hello";

s.size();    // 5 (바이트 수)
s.length();  // 5 (size()와 동일)
s.empty();   // false
s[0];        // 'h' (범위 검사 없음)
s.at(0);     // 'h' (범위 초과 시 예외 발생)
s.front();   // 'h'
s.back();    // 'o'
```

## 문자열 연산

```
std::string a = "hello";
std::string b = " world";

// 연결
std::string c = a + b;    // "hello world"
a += "!";                 // a = "hello!"

// 비교 (사전순)
a == b;    // false
a < b;     // 사전순 비교

// 부분 문자열
std::string s = "hello world";
s.substr(6);    // "world" (6번째부터 끝까지)
s.substr(0, 5); // "hello" (0번째부터 5글자)

// 검색
s.find("world");    // 6 (위치 반환)
s.find("xyz");      // std::string::npos (못 찾을 때)
s.contains("hello"); // true (C++23)
```

# char 배열 vs std::string

## C 스타일 — char 배열

```
1  #include <string.h>
2  #include <stdio.h>
3
4  char s[32] = "hello";
5
6  // 연결 — 버퍼 크기 직접 관리
7  strncat(s, " world", sizeof(s) - strlen(s) - 1);
8
9  // 비교
10 if (strcmp(s, "hello world") == 0) { ... }
11
12 // 길이
13 size_t len = strlen(s);    // O(n)
14
15 // 복사 — 크기 초과 시 버퍼 오버플로
16 strncpy(s, "new text", sizeof(s) - 1);
```

## C++ 스타일 — std::string

```
1  #include <string>
2
3  std::string s = "hello";
4
5  // 연결 — 크기 자동 관리
6  s += " world";
7
8  // 비교 — 연산자로 직관적
9  if (s == "hello world") { ... }
10
11 // 길이
12 size_t len = s.size();    // O(1)
13
14 // 복사 — 안전
15 s = "new text";
```

# std::string — 수정과 변환

C 함수( printf , fopen 등)에 std::string 을 넘길 때는 반드시 .c\_str() 을 호출한다.

## 문자열 수정

```
std::string s = "hello";

// 이어 붙이기
s.append(" world");      // "hello world"
s.push_back('!');        // "hello world!"

// 삽입 / 삭제
s.insert(5, ",");         // "hello, world!"
s.erase(5, 1);           // "hello world!" (5번째 1글자 삭제)

// 교체
s.replace(6, 5, "C++");   // "hello C++!"

// 초기화
s.clear();               // 빈 문자열로
```

## C 문자열과의 변환

```
#include <string>
#include <cstdio>

std::string s = "hello";

// std::string → const char* (C 함수에 넘길 때)
std::printf("%s\n", s.c_str());

// const char* → std::string (자동 변환)
const char* cs = "world";
std::string s2 = cs;      // 복사됨

// 숫자 ↔ 문자열
std::string ns = std::to_string(42);    // "42"
int n = std::stoi("123");               // 123
double d = std::stod("3.14");           // 3.14
```

# const char\* → std::string\_view (C++17)

C의 const char\* 문자열 파라미터를 std::string\_view 로 대체하면 더 안전하고 유연해진다.

## C 스타일 — const char\*

```
#include <cstring>
#include <cstdio>

void print_len(const char* s) {
    if (s == nullptr) return;    // null 체크 필요
    std::printf("길이: %zu\n", std::strlen(s));
}

int main() {
    print_len("hello");

    // std::string을 넘기려면 .c_str() 필요
    std::string name = "홍길동";
    print_len(name.c_str());    // ⚠ 번거로움
}
```

- nullptr 가능 → 방어 코드 필요
- std::string 전달 시 .c\_str() 변환 필요
- 부분 문자열 참조 불가

## C++17 — std::string\_view

```
#include <string_view>
#include <iostream>

// 복사 없이 문자열 참조 — const char*, std::string 모두 수용
void print_len(std::string_view s) {
    std::cout << "길이: " << s.size() << "\n";
}

int main() {
    print_len("hello");           // ✅ const char*
    print_len("world"sv);         // ✅ string_view 리터럴

    std::string name = "홍길동";
    print_len(name);              // ✅ .c_str() 불필요

    // 부분 문자열도 복사 없이 참조
    std::string_view sv = "hello world";
    print_len(sv.substr(0, 5));    // "hello"
}
```

string\_view 는 복사 없는 읽기 전용 뷰다. 원본이 살아 있는 동안에만 유효하다.

# C와 C++의 주요 차이점

같은 코드라도 C 컴파일러와 C++17 컴파일러의 동작이 다를 수 있다.

## void\* 암묵적 변환

```
1 // C: void*는 암묵적으로 변환됨
2 int* p = malloc(sizeof(int)); // ✅ C에서는 OK
3
4 // C++17: 명시적 캐스팅 필요
5 int* p = malloc(sizeof(int)); // ❌ 컴파일 오류
6 int* p = static_cast<int*>(malloc(sizeof(int))); // ✅
```

## bool 타입

```
1 // C99: <stdbool.h> 포함 필요
2 #include <stdbool.h>
3 bool flag = true;
4
5 // C++17: 내장 타입, 헤더 불필요
6 bool flag = true;
```

## const 전역 변수의 링키지

```
1 // C: const 전역은 external linkage (기본)
2 const int MAX = 100; // 다른 파일에서 접근 가능
3
4 // C++17: const 전역은 internal linkage (기본)
5 const int MAX = 100; // 이 파일 내에서만 유효
6
7 // C++17: 헤더에서 여러 파일에 공유하려면
8 inline constexpr int MAX = 100; // ✅ 중복 정의 없음
```

# 혼용 시 주의사항

C 스타일과 C++17 스타일을 한 프로젝트에서 섞을 때 지켜야 할 규칙들이다.

원칙: C 라이브러리를 쓰되, 메모리 관리·타입 안전성은 C++17 방식으로 유지한다.

## malloc / free vs 스마트 포인터

```
// ❌ malloc + delete 혼용 금지
int* p = (int*)std::malloc(sizeof(int));
delete p; // ❌ undefined behavior
```

```
// ❌ new + free 혼용 금지
int* q = new int(42);
std::free(q); // ❌ undefined behavior
```

```
// ✅ 같은 쌍으로만 사용
int* p = (int*)std::malloc(sizeof(int));
std::free(p);
```

```
// ✅ C++17 권장: 스마트 포인터
auto p = std::make_unique<int>(42);
// delete 불필요 - 스코프 종료 시 자동 해제
```

## C++17 코드에서 C 패턴 개선

```
// ❌ C 스타일
void process(const char* input) {
    char buf[256];
    std::strncpy(buf, input, sizeof(buf) - 1);
    buf[sizeof(buf) - 1] = '\0';
    std::printf("처리: %s\n", buf);
}
```

```
// ✅ C++17 스타일
void process(std::string_view input) {
    std::string buf(input); // 크기 안전, 자동 관리
    std::cout << "처리: " << buf << "\n";
}
```

```
// C 구조체 → C++17 유니폼 초기화
struct Point { int x; int y; };
Point p = {3, 4}; // C 집합 초기화 - C++17에서도 동작
Point q{3, 4};    // C++ 유니폼 초기화 - 동일 의미
```

# 요약

```
// C++17 코드에서 C 표준 라이브러리 사용
#include <cstdio>      // printf, scanf, fopen      (C: <stdio.h>)
#include <cstdlib>     // malloc, free, exit        (C: <stdlib.h>)
#include <cstring>     // strcpy, strlen           (C: <string.h>)
#include <cmath>       // sqrt, pow                (C: <math.h>)
```

| 항목                    | C 방식                                      | C++17 권장                                            |
|-----------------------|-------------------------------------------|-----------------------------------------------------|
| 표준 헤더                 | <code>&lt;stdio.h&gt;</code>              | <code>&lt;cstdio&gt;</code>                         |
| null 포인터              | <code>NULL</code>                         | <code>nullptr</code>                                |
| 동적 할당                 | <code>malloc</code> / <code>free</code>   | <code>make_unique</code> / <code>make_shared</code> |
| 타입 캐스팅                | <code>(int)x</code>                       | <code>static_cast&lt;int&gt;(x)</code>              |
| <code>void*</code> 변환 | 암묵적                                       | <code>static_cast&lt;T*&gt;</code> 명시 필요            |
| 문자열 소유·수정             | <code>char[]</code> + <code>strcpy</code> | <code>std::string</code>                            |
| 읽기 전용 문자열 파라미터        | <code>const char*</code>                  | <code>std::string_view</code>                       |